KUBERNETES CLUSTER VERIFICATION & IMPERATIVE COMMANDS

A complete CKA-level checklist for verifying every layer of a kubeadm-installed cluster — from control plane health to networking — plus the 15 imperative commands that solve 70–80% of exam tasks. Built for Kubernetes trainees and sysadmins validating production clusters.

CKA EXAM PREP | KUBERNETES ADMINISTRATION | KUBEADM

VERIFICATION AT EVERY LEVEL

After installing a kubeadm cluster, a disciplined verification routine ensures every subsystem is healthy before workloads are deployed. CKA candidates and sysadmins alike must be comfortable checking all four layers of the stack.

1 — CLUSTER LEVEL

API server endpoint, cluster health, component statuses, namespaces, and resource APIs.

2 — NODE LEVEL

Node readiness, roles, Kubernetes version, CPU/memory capacity, and pod CIDR assignments.

3 — CONTROL PLANE

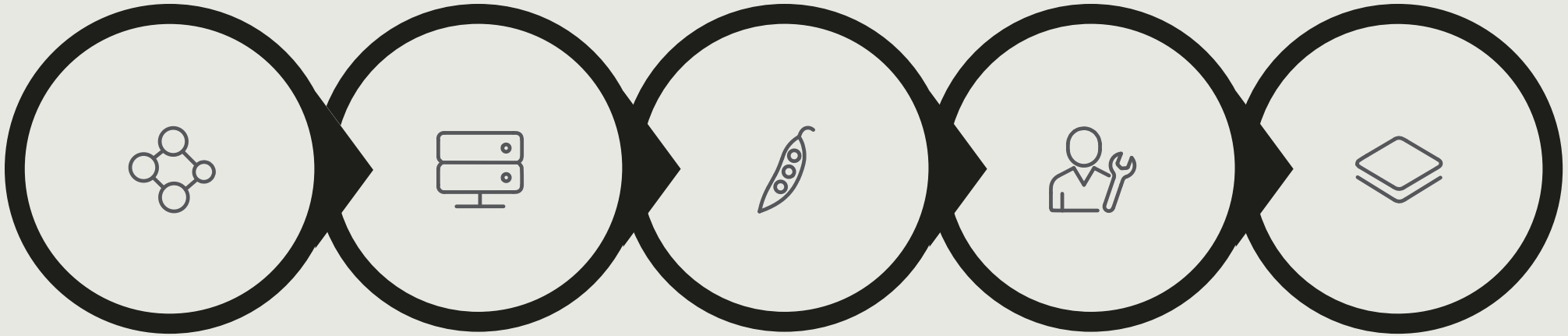
Static pods for API server, etcd, scheduler, and controller manager — logs and statuses.

4 — SYSTEM PODS & NETWORKING

kube-system pods, CNI plugin health (Calico/Flannel), kubelet status, and container runtime.

CLUSTER VERIFICATION CHECKLIST

Work through these commands in order after every kubeadm installation to confirm the cluster is healthy end-to-end.



CLUSTER INFO → **NODE STATUS** → **SYSTEM PODS** → **ALL SERVICES** → **NAMESPACES**

This ordered sequence moves from the highest-level cluster identity check all the way down to the container runtime on each node, giving you full confidence in cluster integrity before deploying workloads.

1 — CHECK CLUSTER INFORMATION

The first two commands confirm the cluster is reachable and that its core components are reporting healthy status. These are the quickest sanity checks after a fresh install.

Command	What It Shows	Example / Notes
`kubectl cluster-info`	API Server endpoint URL, CoreDNS service URL	Kubernetes control plane running at https://10.0.0.10:6443 — confirms API server is reachable
`kubectl get componentstatuses`	Health of Scheduler, Controller Manager, and etcd	Each component shows **Healthy** when the cluster is operating correctly

CKA Tip: `kubectl cluster-info` is the fastest way to confirm the API server endpoint in an exam environment before running any other commands.

2 — CHECK NODES IN THE CLUSTER

Node verification confirms that all control-plane and worker nodes have joined the cluster and are in a **Ready** state. Use `describe` to drill into resource capacity and conditions on any specific node.

Command	What It Shows	Example Output / Notes
`kubectl get nodes`	Node name, STATUS, ROLES, AGE, VERSION	master Ready control-plane 10m v1.29 / worker1 Ready <none> 5m v1.29
`kubectl describe node master`	CPU, Memory, Pod CIDR, Conditions, Allocatable resources, Taints	Look for **MemoryPressure=False**, **DiskPressure=False**, and **Ready=True** in Conditions

3 — CHECK SYSTEM PODS IN KUBE-SYSTEM

Kubernetes runs all control plane and networking components as pods inside the `kube-system` namespace. Every pod here must show **Running** status for the cluster to function correctly.

Command	What It Shows	When to Use
`kubectl get pods -n kube-system`	All system pods: coredns, etcd-master, kube-apiserver-master, kube-controller-manager-master, kube-scheduler-master, kube-proxy	First check after install; after any upgrade
`kubectl get pods -n kube-system -w`	Continuous live watch — shows state transitions in real time	During restarts or upgrades when pods are coming up

4 — CONTROL PLANE COMPONENTS

Control plane components run as **static pods** managed directly by kubelet from manifest files in `/etc/kubernetes/manifests`. Understanding each component's role is essential for CKA troubleshooting questions.

Component Pod Name	Purpose	Log Command
`kube-apiserver-master`	Entry point to the cluster — all kubectl commands hit this first	`kubectl logs kube-apiserver-master -n kube-system`
`etcd-master`	Distributed key-value store — the cluster's source of truth	`kubectl logs etcd-master -n kube-system`
`kube-scheduler-master`	Assigns newly created pods to appropriate nodes	`kubectl logs kube-scheduler-master -n kube-system`
`kube-controller-manager-master`	Reconciles actual vs. desired cluster state continuously	`kubectl logs kube-controller-manager-master -n kube-system`

5 & 6 — SERVICES AND NAMESPACES

CHECK ALL SERVICES

```
kubectl get svc -A
```

Example output:

NAMESPACE	NAME	TYPE / CLUSTER-IP
default	kubernetes	ClusterIP 10.96.0.1
kube-system	kube-dns	ClusterIP 10.96.0.10

Confirms DNS and API server services are exposed at the correct cluster IPs.

CHECK ALL NAMESPACES

```
kubectl get ns
```

Default namespaces present in every kubeadm cluster:

- **default** — where user workloads run by default
- **kube-system** — all system and control plane components
- **kube-public** — publicly readable; stores cluster info
- **kube-node-lease** — stores node heartbeat lease objects

Missing namespaces or a kube-dns service with no IP are early signs of a broken install.

7 — CHECK NETWORKING (CNI PLUGIN)

Kubernetes networking requires a Container Network Interface (CNI) plugin to be installed. Without a healthy CNI, pods cannot communicate across nodes and will remain in a **Pending** state indefinitely.

CNI Plugin	Pods to Look For	Verification Command
Calico	calico-node, calico-kube-controllers	`kubectl get pods -n kube-system | grep calico`
Flannel	kube-flannel	`kubectl get pods -n kube-system | grep flannel`
Weave	weave-net	`kubectl get pods -n kube-system | grep weave`

Key Rule: If nodes are in **NotReady** status after installation, the CNI plugin is almost always the cause. Install it immediately after kubeadm init.

8, 9 & 10 — KUBELET, CONTAINER RUNTIME & STATIC MANIFESTS

These three checks operate below the Kubernetes API layer — directly on the node OS. They are critical for diagnosing issues that `kubectl` cannot surface.

Check	Command	What to Look For
kubelet service status	`systemctl status kubelet`	Status must be **active (running)**. If stopped, control plane components and static pods will not start.
kubelet process	`ps -ef | grep kubelet`	Confirm kubelet is running with the correct `--config` and `--kubeconfig` flags.
Container runtime (containerd)	`crictl ps`	Lists running containers: kube-apiserver, kube-controller-manager, kube-scheduler, etcd should all appear.
Static pod manifests	`ls /etc/kubernetes/manifests`	Must contain: etcd.yaml, kube-apiserver.yaml, kube-controller-manager.yaml, kube-scheduler.yaml

11 — CHECK KUBECONFIG

The kubeconfig file defines which cluster, user, and namespace the `kubectl` CLI targets. Misconfigured contexts are a common source of confusion in multi-cluster CKA exam scenarios.

Command	Purpose
`ls ~/.kube/config`	Confirm the kubeconfig file exists at the default location used by kubectl
`kubectl config view`	Display full kubeconfig: clusters, users, contexts, and server URLs
`kubectl config current-context`	Show the active context — confirms which cluster kubectl is currently targeting
`kubectl config use-context <name>`	Switch to a different cluster context — essential in multi-cluster CKA questions

CKA Exam Tip: Every CKA exam question specifies a context. Always run `kubectl config use-context <context-name>` at the start of each question to avoid working on the wrong cluster.

12 & 13 — FULL RESOURCE DISCOVERY & GET EVERYTHING

DISCOVER ALL API RESOURCES

```
kubectl api-resources
```

Lists every resource type Kubernetes supports — pods, deployments, services, configmaps, CRDs, and more — along with their API group, namespaced status, and short names. Indispensable when you can't remember exact resource names during the exam.

GET EVERYTHING ACROSS ALL NAMESPACES

```
kubectl get all -A
```

A single command that returns pods, services, deployments, ReplicaSets, StatefulSets, DaemonSets, and Jobs across every namespace. Use this for a fast cluster-wide health snapshot. Combine with `-o wide` to also see node placement and pod IPs.

> This is the single most powerful read-only command for a cluster overview.

14 — TROUBLESHOOTING COMMANDS

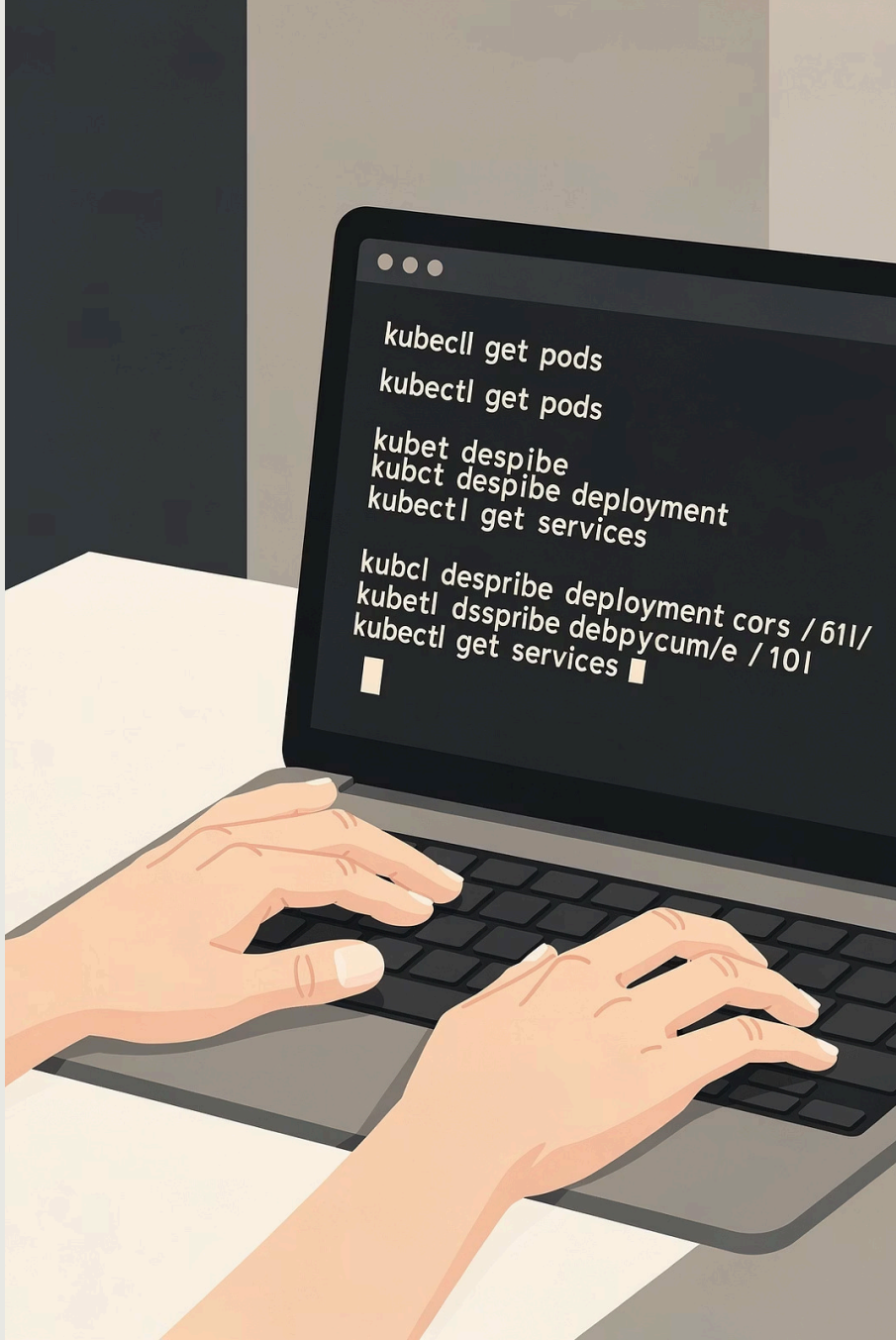
These two commands are among the most frequently tested in CKA scenarios involving broken or misbehaving resources. Master them before exam day.

Command	What It Shows	CKA Use Case
`kubectl get events -A`	All cluster events across every namespace — warnings, failures, scheduling decisions	First step when a pod won't start; reveals FailedScheduling, OOMKilled, ImagePullBackOff causes
`kubectl describe pod <name>`	Full pod spec, current conditions, container state, resource requests/limits, and event history	Diagnose CrashLoopBackOff, pending pods, or wrong image names in exam tasks
`kubectl get pods -A -o wide`	Every pod with Node, IP, STATUS, and READY columns visible at once	Best beginner overview — teaches trainees to correlate pod placement with node health

CHAPTER 2

15 IMPERATIVE COMMANDS THAT WIN THE CKA EXAM

For CKA exam and real Kubernetes work, approximately **15 imperative commands** solve 70–80% of all exam tasks quickly. These commands span the full lifecycle: Pod → Deployment → Service → Troubleshooting → YAML generation. Students must memorize and practice these until they are second nature.



POD COMMANDS (1–4): CREATE, LABEL, PORT & YAML

Pod creation commands form the foundation of CKA work. The `--dry-run=client -o yaml` pattern is the single most time-saving technique in the entire exam — use it to generate YAML instantly instead of writing from scratch.

Command	Purpose	Verify / Notes
`kubectl run nginx --image=nginx`	Create a pod named nginx	`kubectl get pods`
`kubectl run nginx --image=nginx --labels app=web`	Create pod with label `app=web`	`kubectl get pods --show-labels`
`kubectl run nginx --image=nginx --port=80`	Create pod exposing port 80 — required before using `kubectl expose`	`kubectl describe pod nginx`
`kubectl run nginx --image=nginx --dry-run=client -o yaml > pod.yaml`	Generate pod YAML without creating the resource — edit then apply	`kubectl apply -f pod.yaml`

Golden Rule: Always use `--dry-run=client -o yaml` to generate YAML. Saves 2–3 minutes per task in the exam.

DEPLOYMENT COMMANDS (5–7): CREATE, REPLICAS & SCALE

Deployments manage ReplicaSets and ensure the desired number of pod replicas are always running. Scaling is one of the most common CKA tasks — practice it until it takes under 10 seconds.

Command	Purpose	Verify
`kubectl create deployment nginx --image=nginx`	Create a deployment with 1 replica (default)	`kubectl get deployments`
`kubectl create deployment web --image=nginx --replicas=3`	Create deployment with 3 replicas immediately	`kubectl get pods` – expect 3 pods
`kubectl scale deployment web --replicas=5`	Scale an existing deployment up or down imperatively	`kubectl get pods` – expect 5 pods

Use `kubectl create deployment web --image=nginx --replicas=3 --dry-run=client -o yaml` to generate deployment YAML with replica count pre-set.

SERVICE COMMANDS (8–10): EXPOSE POD, DEPLOYMENT & YAML

Exposing workloads as Services is tested in nearly every CKA exam. Both `NodePort` and `ClusterIP` types appear frequently. Use `--dry-run=client -o yaml` when you need to customize port mappings or selectors before creating the service.

Command	Purpose	Verify
`kubectl expose pod nginx --port=80 --type=NodePort`	Expose a pod directly as a NodePort service	`kubectl get svc`
`kubectl expose deployment web --port=80 --type=NodePort`	Expose all pods in a deployment via a NodePort service	`kubectl get svc`
`kubectl expose deployment web --port=80 --type=NodePort --dry-run=client -o yaml > svc.yaml`	Generate service YAML for customization — edit targetPort or nodePort before applying	`kubectl apply -f svc.yaml`

OPERATIONS COMMANDS (11–15): EDIT, DESCRIBE, GET ALL, LOGS & EXEC

These five commands cover the full lifecycle of operating and debugging live Kubernetes resources. Together they give you complete visibility and control over any workload in the cluster.

Command	Purpose	Key Details
`kubectl edit deployment web`	Live-edit any resource spec in your default editor	Change image, replicas, env vars — saves and applies immediately on exit
`kubectl describe pod nginx`	Full resource details and event history	Shows errors, container status, resource limits, scheduling decisions
`kubectl get all`	List pods, services, deployments, replicasets in default namespace	Add `-A` for all namespaces
`kubectl logs nginx` / `kubectl logs pod -c container`	Stream container stdout/stderr logs	Use `-c` flag for multi-container pods
`kubectl exec -it nginx -- /bin/bash`	Open interactive shell inside a running pod	Run `curl localhost` to verify web server is responding

BONUS COMMANDS: WATCH, WIDE OUTPUT & DELETE

WATCH & WIDE OUTPUT

```
kubectl get pods -w
```

Live-streams pod status changes — invaluable during rolling deployments or when waiting for a pod to reach Running state.

```
kubectl get pods -o wide
```

Adds NODE, IP, and NOMINATED NODE columns — essential for verifying pod placement across nodes and diagnosing node affinity issues.

DELETE RESOURCES

```
kubectl delete pod nginx
```

Imperatively delete a pod, deployment, service, or any resource by type and name. For faster deletion in exams use `--grace-period=0 --force` on stuck pods.

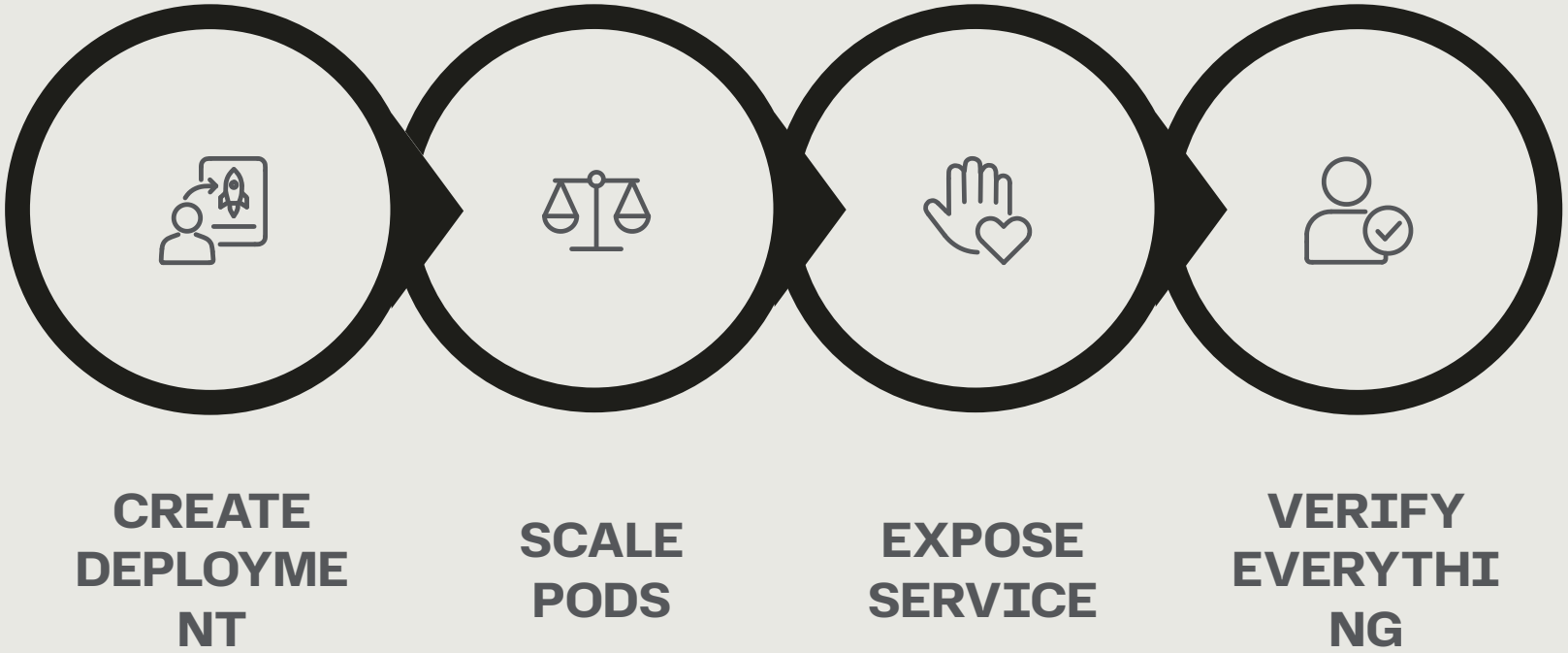
APPLY FROM FILE

```
kubectl apply -f pod.yaml
```

The idempotent way to create or update resources from YAML. Works for first-time creates and subsequent updates — the standard production workflow.

REAL CKA EXAM WORKFLOW — END TO END

This is the exact sequence to execute when an exam question asks you to deploy a scaled web application and expose it. Practice until this flows in under 2 minutes.



Each step builds on the last. After `kubectl get all` confirms the deployment, replicaset, pods, and service are all present and healthy — the task is complete. This four-step pattern solves the majority of CKA deployment questions.

THE GOLDEN TRICK: `--DRY-RUN=CLIENT -O YAML`

This single flag combination is the most impactful time-saver in the entire CKA exam. Instead of writing YAML manifests from scratch — which is slow and error-prone — generate them instantly from imperative commands and customize as needed.

GENERATE POD YAML

```
kubectl run nginx --image=nginx \
  --dry-run=client -o yaml > pod.yaml
```

Instantly produces a valid pod manifest. Edit resources, env vars, or labels — then `kubectl apply -f pod.yaml`.

GENERATE DEPLOYMENT YAML

```
kubectl create deployment nginx \
  --image=nginx \
  --dry-run=client -o yaml
```

Add `--replicas=3` before `--dry-run` to pre-populate replica count in the output YAML.

GENERATE SERVICE YAML

```
kubectl expose deployment web \
  --port=80 --type=NodePort \
  --dry-run=client -o yaml > svc.yaml
```

Customize `nodePort` or `targetPort` in the YAML before applying — not possible with a direct imperative command.

GOLDEN COMMANDS STUDENTS MUST MEMORIZE

These 12 commands form the core vocabulary every CKA candidate must know by heart. Fluency with these commands — their flags, outputs, and common use cases — is the foundation for exam success and real-world cluster administration.

CREATE RESOURCES

- kubectl run
- kubectl create deployment
- kubectl expose
- kubectl apply -f

MODIFY RESOURCES

- kubectl scale
- kubectl edit
- kubectl delete

OBSERVE & DEBUG

- kubectl logs
- kubectl exec
- kubectl describe
- kubectl get all

COMPLETE COMMAND REFERENCE AT A GLANCE

A comprehensive quick-reference table covering both cluster verification and imperative commands — organized by category for fast lookup during study sessions and exam preparation.

Category	Command	Purpose
Cluster Info	`kubectl cluster-info`	API server endpoint and CoreDNS URL
Cluster Health	`kubectl get componentstatuses`	Scheduler, controller manager, etcd health
Nodes	`kubectl get nodes`	All nodes with status, role, version
Nodes	`kubectl describe node <name>`	CPU, memory, pod CIDR, conditions
System Pods	`kubectl get pods -n kube-system`	All control plane and networking pods
Services	`kubectl get svc -A`	All services across all namespaces
Namespaces	`kubectl get ns`	All namespaces in the cluster
Events	`kubectl get events -A`	Cluster-wide warnings and scheduling events
Pods (create)	`kubectl run nginx --image=nginx`	Imperatively create a pod
YAML gen	`kubectl run nginx --image=nginx --dry-run=client -o yaml`	Generate pod YAML without creating resource
Deployment	`kubectl create deployment web --image=nginx --replicas=3`	Create deployment with replica count
Scale	`kubectl scale deployment web --replicas=5`	Imperatively scale deployment
Service	`kubectl expose deployment web --port=80 --type=NodePort`	Expose deployment as NodePort service
Logs	`kubectl logs <pod> -c <container>`	Container logs; use -c for multi-container pods
Debug	`kubectl exec -it <pod> -- /bin/bash`	Interactive shell inside running pod
Overview	`kubectl get all -A`	Everything across all namespaces
Discovery	`kubectl api-resources`	All supported resource types and API groups
kubeconfig	`kubectl config current-context`	Confirm active cluster context
Runtime	`crictl ps`	Running containers via container runtime
Node agent	`systemctl status kubelet`	kubelet service health on each node

KEY TAKEAWAYS FOR CKA CANDIDATES

Master these principles and you will navigate the CKA exam with speed and confidence.

1 ALWAYS VERIFY CONTEXT FIRST

Run `kubectl config current-context` at the start of every exam question. Working on the wrong cluster is an easy way to lose points.

2 USE `--DRY-RUN=CLIENT -O YAML` FOR EVERYTHING

Never write YAML from scratch. Generate it imperatively, redirect to a file, edit, and apply. This is faster and more accurate under exam pressure.

3 CHECK SYSTEM PODS WHEN NODES MISBEHAVE

Nodes in NotReady state almost always point to a CNI issue or a crashed control plane pod. `kubectl get pods -n kube-system` and `systemctl status kubelet` will surface the cause.

4 EVENTS AND DESCRIBE ARE YOUR DEBUGGERS

`kubectl get events -A` and `kubectl describe pod` together will diagnose 90% of broken pod scenarios — ImagePullBackOff, OOMKilled, FailedScheduling, and more.